

Laporan Tugas Kecil 1 IF2211 Strategi Algoritma  
Semester II Tahun 2025/2026  
Penyelesaian Queens LinkedIn

## I. Pendahuluan

### A. Latar Belakang

Permainan *Queens* pada platform LinkedIn merupakan pengembangan dari masalah klasik *N-Queens* yang sering dipelajari dalam materi strategi algoritma. Pada permainan ini, pemain harus menempatkan *Queen* pada papan berukuran  $N \times N$  dengan beberapa aturan, yaitu setiap baris, kolom, dan setiap daerah warna hanya boleh berisi satu *Queen*. Selain itu, *Queen* juga tidak boleh saling bersebelahan, termasuk secara diagonal.

Karena banyaknya aturan yang harus dipenuhi secara bersamaan, tidak semua susunan *Queen* dapat menjadi solusi yang valid. Seiring bertambahnya ukuran papan, jumlah kemungkinan konfigurasi juga meningkat secara signifikan sehingga proses pencarian solusi menjadi lebih menantang. Oleh karena itu, diperlukan suatu metode yang dapat mencoba berbagai kemungkinan penempatan secara sistematis.

Pada tugas ini digunakan algoritma brute force untuk mencari solusi. Pendekatan ini dilakukan dengan mencoba semua kemungkinan konfigurasi yang ada tanpa menggunakan heuristik tambahan. Dengan cara ini, proses pencarian dapat diamati secara langsung, termasuk beberapa banyak konfigurasi yang diperiksa dan berapa lama waktu yang dibutuhkan untuk menemukan solusi atau menyimpulkan bahwa tidak ada solusi.

### B. Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam tugas ini adalah:

1. Bagaimana merancang algoritma brute force untuk menemukan solusi permainan *Queens* yang memenuhi seluruh constraint?
2. Bagaimana memastikan bahwa konfigurasi yang dihasilkan valid terhadap aturan baris, kolom, daerah warna, dan larangan bersebelahan?
3. Bagaimana menghitung jumlah konfigurasi yang ditinjau selama proses pencarian?
4. Bagaimana mengukur waktu eksekusi algoritma brute force dalam menyelesaikan permasalahan?

### C. Tujuan

Tujuan dari pelaksanaan tugas ini adalah :

1. Mengimplementasikan algoritma brute force untuk menyelesaikan permainan *Queens* sesuai dengan spesifikasi yang diberikan.
2. Menghasilkan solusi yang valid atau menyatakan bahwa tidak terdapat solusi.
3. Menghitung dan menampilkan jumlah konfigurasi yang ditinjau selama proses pencarian.
4. Mengukur waktu eksekusi algoritma pencarian solusi.
5. Menganalisis performa algoritma brute force terhadap ukuran papan yang berbeda.

## II. Algoritma Brute Force

### A. Ide Dasar Algoritma

Pada tugas ini, permainan *Queens* diselesaikan menggunakan metode brute force. Artinya, program akan mencoba semua kemungkinan peletakan *queen* yang mungkin sampai menemukan konfigurasi yang memenuhi semua aturan. Tidak ada heuristik atau optimasi khusus yang digunakan sehingga pendekatan ini benar-benar mengandalkan pemeriksaan satu per satu terhadap semua kombinasi yang ada.

Jumlah *Queen* yang harus ditempatkan sama dengan jumlah warna unik pada papan. Jika papan berukuran  $N \times N$ , maka total sel yang tersedia adalah  $N^2$ . Program kemudian mencoba memilih sejumlah posisi dari seluruh sel tersebut dan mengecek apakah kombinasi tersebut memenuhi aturan permainan. Proses ini terus dilakukan sampai solusi ditemukan atau semua kemungkinan sudah dicoba.

### B. Langkah-langkah Algoritma

Program dimulai dengan meminta pengguna memasukkan nama file input. Setelah itu, File tersebut akan dibaca dan disimpan ke dalam `vector<string>` menggunakan fungsi pembacaan file. Dari jumlah baris yang terbaca, program menentukan ukuran papan. Kemudian, menghitung jumlah warna unik yang terdapat pada papan dengan memasukkan setiap karakter ke dalam struktur set. Jumlah warna unik ini menentukan berapa banyak *queen* yang harus ditempatkan.

Setelah informasi dasar papan diketahui, program akan menampilkan ukuran papan, jumlah warna, serta jumlah kombinasi yang mungkin dicoba. Program kemudian menunggu input pengguna untuk mulai menjalankan proses pencarian solusi.

Proses pencarian dimulai dengan mencatat waktu awal menggunakan fungsi pengukuran waktu dari library chrono. Selanjutnya, fungsi `bruteForceAlgorithm()` dipanggil. di dalam fungsi ini, program akan membentuk kombinasi awal sejumlah K, dimana K adalah jumlah warna unik, posisi dari

seluruh sel yang tersedia. Setiap kombinasi direpresentasikan dalam bentuk indeks linear yang kemudian dikonversi menjadi pasangan koordinat (baris, kolom).

Untuk setiap kombinasi yang terbentuk, program menambahkan penghitung iterasi sebagai tanda bahwa satu konfigurasi telah diperiksa. Setelah itu, kombinasi tersebut diuji menggunakan fungsi `isValid()`. Fungsi ini memeriksa apakah ada konflik pada baris, kolom, warna, maupun aturan bersebelahan. Jika konfigurasi valid, proses pencarian langsung dihentikan dan solusi dikembalikan ke fungsi utama. Jika tidak valid, program membangkitkan kombinasi berikutnya menggunakan fungsi `nextCombination()` dan proses yang sama diulang kembali.

Proses ini terus berjalan sampai solusi ditemukan atau seluruh kombinasi telah dicoba. Jika semua kombinasi habis dan tidak ada yang valid, fungsi akan mengembalikan hasil kosong sebagai tanda bahwa solusi tidak ada.

Setelah proses brute force selesai, program mencatat waktu akhir dan menghitung selisih waktu dalam satuan milidetik. Hasil kemudian ditampilkan di terminal, termasuk papan solusi (jika ada), waktu pencarian, dan jumlah konfigurasi yang telah diperiksa. Jika solusi ditemukan, pengguna diberi pilihan untuk menyimpan hasil ke dalam file baru dengan nama yang ditambahkan `_solution`. Terakhir, pengguna dapat memilih untuk mencoba file lain atau mengakhiri program.

#### C. Kompleksitas Waktu

Kompleksitas waktu dari algoritma ini bergantung pada jumlah kombinasi yang harus diperiksa. Jika papan berukuran  $N \times N$  dan terdapat  $K$  warna unik, maka program akan mencoba seluruh kemungkinan pemilihan  $K$  posisi dari total  $N^2$  sel. Jumlah kemungkinan tersebut secara matematis dapat dinyatakan sebagai :

$$C(N^2, K)$$

Karena pendekatan yang digunakan adalah brute force murni, dalam kasus terburuk program harus memeriksa seluruh kombinasi tersebut sebelum menemukan solusi atau menyimpulkan bahwa solusi tidak ada.

Untuk setiap kombinasi yang diperiksa, program masih harus melakukan proses validasi. pemeriksaan baris, kolom, dan warna memerlukan waktu yang sebanding dengan jumlah queen ( $K$ ). Selain itu, pemeriksaan aturan bersebelahan membandingkan setiap pasangan queen sehingga memerlukan waktu sekitar  $O(K^2)$ .

Dengan demikian, secara kasar kompleksitas total algoritma dapat diperkirakan sebagai :

$$O(C(N^2, K) \cdot K^2)$$

Nilai kombinasi  $C(N^2, K)$  dapat bertambah cepat seiring meningkatnya ukuran papan. Oleh karena itu, waktu eksekusi juga meningkat secara signifikan untuk papan yang lebih besar. Hal ini menunjukkan bahwa metode brute force kurang efisien untuk ukuran besar, namun tetap sesuai dengan tujuan tugas karena seluruh ruang solusi diperiksa tanpa menggunakan optimasi tambahan.

### III. Implementasi Program

#### A. Struktur Folder Repository

```

├── src
│   ├── helperfunctions.hpp
│   ├── main.cpp
│   └── helperfunctions.cpp
├── obj
│   ├── main.o
│   └── helperfunctions.o
├── README.md
├── bin
│   ├── lmao.txt
│   └── queens
├── test
│   ├── test1.txt
│   ├── test1_solution.txt
│   ├── tc4.txt
│   ├── tc1.txt
│   ├── tc2.txt
│   ├── tc3.txt
│   ├── tc5.txt
│   ├── tc1_solution.txt
│   ├── tc2_solution.txt
│   ├── tc3_solution.txt
│   └── tc5_solution.txt
└── docs
    ├── lmao.txt
    └── LaporanStima_Tucill1_13524078.pdf

```

Repository program ini disusun dengan tujuan agar seluruh komponen program terorganisir dengan rapi dan mudah dipahami. Struktur folder dipisahkan berdasarkan fungsi masing-masing bagian. Folder `src` berisi seluruh source code utama program, yaitu `main.cpp`, `helperfunction.cpp`, dan `helperfunction.hpp`. File `main.cpp` berfungsi sebagai pusat kontrol program, sedangkan `helperfunctions.cpp` berisi implementasi fungsi-fungsi pendukung yang digunakan oleh program utama, dan `helperfunctions.hpp` berisi deklarasi fungsi agar dapat digunakan lintas file.

Folder `test` digunakan untuk menyimpan file input yang akan diuji oleh program. File-file dalam folder ini berisi konfigurasi papan awal yang akan diproses oleh algoritma. Apabila solusi ditemukan dan pengguna memilih untuk menyimpannya maka file solusi akan di folder yang sama dengan tambahan `_solution` pada nama file. Folder `obj` berisi file object hasil kompilasi sementara sebelum digabung menjadi executable. Sementara itu, folder `bin` menyimpan file executable utama yang dapat langsung dijalankan oleh pengguna. Struktur ini dibuat agar program memiliki pemisahan yang jelas antara komponen sehingga lebih mudah dalam proses pengembangan, pengujian, dan pemeliharaan.

## B. Source Code

Program ini dibagi menjadi dua bagian utama, yaitu file `main.cpp` yang mengatur alur utama program dan file `helperfunctions.cpp` yang berisi seluruh implementasi fungsi pendukung. Pemisahan ini dilakukan agar kode lebih modular dan tidak menumpuk dalam satu file besar sehingga memudahkan pembacaan dan debugging.

### 1. `refreshScreen()`

```
PROCEDURE refreshScreen
    IF system is Windows THEN
        execute command "cls"
    ELSE
        execute command "clear"
    ENDIF
ENDPROCEDURE
```

Fungsi ini digunakan untuk membersihkan tampilan layar terminal sebelum program menampilkan output baru. Tujuan utama dari fungsi ini adalah menjaga tampilan tetap rapih, terutama karena program dijalankan di dalam `loop` sehingga proses input dan output dapat terjadi berkali-kali.

Secara Implementasi, fungsi ini memanggil perintah sistem untuk membersihkan layar. Perintah yang digunakan bergantung pada sistem operasi yang digunakan, misalnya `cls` untuk *Windows* dan `clear` untuk sistem berbasis *Unix* seperti *Linux* atau *macOS*. Dengan adanya fungsi ini, tampilan program tidak menumpuk hasil eksekusi sebelumnya sehingga pengalaman pengguna menjadi lebih nyaman.

### 2. `Faktorial()`

```
FUNCTION Faktorial(n : integer) → long long
    hasil ← 1

    FOR i ← 2 TO n DO
        hasil ← hasil × i
    ENDFOR

    RETURN hasil
ENDFUNCTION
```

Fungsi ini menghitung nilai faktorial dari suatu bilangan bulat  $n$ , yang secara matematis didefinisikan sebagai :

$$n! = n \times n(n - 1) \times (n - 2) \times \dots \times 1$$

Konsep algoritma ini berasal dari definisi matematis faktorial dalam kombinatorika dasar. Implementasi iteratif dipilih untuk menghindari *overhead* pemanggilan fungsi berulang dan potensi *stack overflow*.

### 3. `Combination()`

```
FUNCTION Combination(n : integer, r : integer) → long long

    IF r > n OR r < 0 THEN
        RETURN 0
```

```

ENDIF

IF r = 0 OR r = n THEN
    RETURN 1
ENDIF

IF r > n / 2 THEN
    r ← n - r
ENDIF

result ← 1

FOR i ← 0 TO r - 1 DO
    result ← result * (n - i)
    result ← result DIV (i + 1)
ENDFOR

RETURN result
ENDFUNCTION

```

Fungsi ini menghitung nilai kombinasi  $C(n, r)$ , yaitu jumlah cara memilih  $r$  elemen dari  $n$  elemen tanpa memperhatikan urutan. Secara matematis :

$$C(n, r) = \frac{n!}{(r!(n-r)!)}$$

Namun, implementasi dalam program tidak langsung menggunakan rumus faktorial penuh karena nilai faktorial dapat menjadi sangat besar dan menyebabkan *overflow*. Sebagai gantinya, digunakan pendekatan iteratif berbasis sifat berikut :

$$C(n, r) = \frac{n \times (n-1) \times \dots \times (n-r+1)}{r!}$$

Selain itu, digunakan sifat simetri kombinasi :

$$C(n, r) = C(n, n - r)$$

Sehingga jika  $r > n/2$ , maka  $r$  diganti menjadi  $n - r$  untuk mengurangi jumlah iterasi.

#### 4. ReadFileToArray()

```

PROCEDURE ReadFileToArray(filename, isiFile, returnArraySize)

    OPEN file with filename
    size ← 0

    IF file is open THEN
        WHILE there is a line to read DO
            READ line
            APPEND line to isiFile
            size ← size + 1
        ENDWHILE

        CLOSE file
        *returnArraySize ← size
    ELSE
        PRINT "Can't found any files!"
    ENDIF

ENDPROCEDURE

```

Fungsi ini bertanggung jawab membaca file input dan menyimpannya ke dalam struktur data `vector<string>`.

Proses pembacaan dilakukan menggunakan `ifstream` dan `getline()` untuk membaca file baris demi baris. Setiap baris disimpan sebagai satu elemen dalam `vector`. Setelah seluruh file terbaca, jumlah elemen `vector` digunakan untuk menentukan ukuran papan  $N$ .

#### 5. nextCombination()

```

FUNCTION nextCombination(strava, N) → boolean

    track ← size of strava

```

```

FOR i ← track - 1 DOWNT0 0 DO
  IF strava[i] < N - track + i THEN

    strava[i] ← strava[i] + 1

    FOR j ← i + 1 TO track - 1 DO
      strava[j] ← strava[j - 1] + 1
    ENDFOR

    RETURN TRUE
  ENDIF
ENDFOR

RETURN FALSE
ENDFUNCTION

```

Fungsi ini merupakan implementasi algoritma pembangkit kombinasi berikutnya. Fungsi ini digunakan untuk menghasilkan semua kombinasi  $r$  elemen dari total  $n$  elemen tanpa duplikasi.

Cara kerja :

- Mencari indeks paling kanan yang masih dapat dinaikkan nilainya
- Menaikkan nilai elemen tersebut
- Mengatur ulang semua elemen setelahnya agar tetap berurutan naik

Jika tidak ada lagi kombinasi yang dapat dibentuk, fungsi mengembalikan false.

Pendekatan ini sering digunakan dalam *brute force combinatorial generation* karena efisien dan menghasilkan semua subset berukuran tetap tanpa perlu memeriksa seluruh  $2^n$  kemungkinan.

6. isValid()

```

FUNCTION isValid(QueenPositions, N, isiFile) → boolean

  size ← size of QueenPositions

  DECLARE colorUsed[26] initialized to FALSE
  DECLARE rowUsed[N] initialized to FALSE
  DECLARE colUsed[N] initialized to FALSE

  FOR i ← 0 TO size - 1 DO

    baris ← QueenPositions[i].first
    kolom ← QueenPositions[i].second
    warna ← isiFile[baris][kolom]

    colorIdx ← warna - 'A'

    IF colorUsed[colorIdx] = TRUE THEN
      RETURN FALSE
    ENDIF
    colorUsed[colorIdx] ← TRUE

    IF rowUsed[baris] = TRUE THEN
      RETURN FALSE
    ENDIF
    rowUsed[baris] ← TRUE

    IF colUsed[kolom] = TRUE THEN
      RETURN FALSE
    ENDIF
    colUsed[kolom] ← TRUE

  ENDFOR

  FOR i ← 0 TO size - 1 DO
    baris1 ← QueenPositions[i].first
    kolom1 ← QueenPositions[i].second

    FOR j ← i + 1 TO size - 1 DO
      baris2 ← QueenPositions[j].first
      kolom2 ← QueenPositions[j].second
    
```

```

        IF |baris1 - baris2| ≤ 1 AND |kolom1 - kolom2| ≤ 1 THEN
            RETURN FALSE
        ENDIF
    ENDFOR
ENDFOR

RETURN TRUE
ENDFUNCTION

```

Fungsi ini digunakan untuk memeriksa apakah suatu konfigurasi posisi *queen* memenuhi seluruh aturan yang telah ditentukan. Fungsi ini hanya berperan sebagai mekanisme validasi terhadap satu kandidat solusi yang dihasilkan oleh algoritma brute force. Proses validasi dilakukan dalam dua tahap utama.

Tahap pertama adalah pengecekan terhadap konflik baris, kolom, dan warna. Untuk memastikan tidak ada dua *queen* berada pada baris yang sama, digunakan struktur `vector<bool>` yang menandai apakah suatu baris sudah terisi *queen*. Mekanisme yang sama digunakan untuk kolom. Selain itu, digunakan *array boolean* untuk mencatat warna yang sudah digunakan sehingga tidak ada dua *queen* yang ditempatkan pada sel dengan warna yang sama. Jika salah satu kondisi ini dilanggar, fungsi langsung mengembalikan nilai `false`.

Tahap kedua adalah pengecekan terhadap konflik jarak antar *queen*. Pada bagian ini, setiap pasangan *queen* dibandingkan satu per satu menggunakan dua buah perulangan bersarang. Selisih baris dan kolom dihitung menggunakan nilai absolut. Jika terdapat dua *queen* yang memiliki jarak terlalu dekat atau berada dalam area yang tidak diperbolehkan, maka konfigurasi dianggap tidak valid dan fungsi mengembalikan `false`.

Apabila seluruh pengecekan berhasil dilewati tanpa pelanggaran, maka fungsi mengembalikan nilai `true` yang menandakan bahwa konfigurasi tersebut memenuhi seluruh aturan.

7. `bruteForceAlgorithm()`

```

FUNCTION bruteForceAlgorithm(isiFile, N, totalWarna, countIterasi)
    totalSel ← N × N

    DECLARE strava with size totalWarna

    FOR i ← 0 TO totalWarna - 1 DO
        strava[i] ← i
    ENDFOR

    DECLARE QueenPositions

    printTime ← current time
    print_interval_ms ← 5000

    WHILE TRUE DO

        countIterasi ← countIterasi + 1
        CLEAR QueenPositions

        FOR each idx IN strava DO
            ADD (idx DIV N, idx MOD N) TO QueenPositions
        ENDFOR

        currentTime ← current time
        elapsed ← time difference between currentTime and printTime

        IF elapsed ≥ print_interval_ms THEN
            CALL refreshScreen
            DISPLAY progress
            CALL printTiles(isiFile, QueenPositions, N)
            printTime ← currentTime
        ENDIF

        IF isValid(QueenPositions, N, isiFile) THEN
            CALL refreshScreen
            DISPLAY solution found
            RETURN QueenPositions
        ENDIF

        IF nextCombination(strava, totalSel) = FALSE THEN

```

```

        BREAK
    ENDIF

ENDWHILE

RETURN empty vector
ENDFUNCTION

```

Fungsi `bruteForceAlgorithm()` merupakan inti utama dari program karena di dalamnya proses pencarian solusi dilakukan. Fungsi ini menerapkan pendekatan *brute force* dengan cara menghasilkan seluruh kemungkinan kombinasi posisi *queen* berdasarkan jumlah warna unik yang harus ditempatkan pada papan. Setiap kombinasi yang dihasilkan akan diperiksa apakah memenuhi seluruh aturan yang telah ditentukan.

Pada awal fungsi, jumlah total sel pada papan dihitung sebagai  $N \times N$ . Nilai ini merepresentasikan seluruh kemungkinan posisi yang tersedia. Selanjutnya, dibuat sebuah `vector integer` yang berisi indeks awal kombinasi sebanyak `jumlahWarna`. Indeks tersebut merepresentasikan posisi sel dalam bentuk satu dimensi yang nantinya akan dikonversi menjadi koordinat dua dimensi berupa baris dan kolom. Representasi satu dimensi ini digunakan untuk mempermudah proses pembangkitan kombinasi.

Proses pencarian dilakukan dalam sebuah perulangan yang terus berjalan selama masih terdapat kombinasi yang dapat dibentuk. Pada setiap iterasi, penghitung jumlah iterasi akan ditambah untuk mencatat berapa banyak konfigurasi yang telah diperiksa. Kombinasi indeks yang sedang aktif kemudian dikonversi menjadi koordinat dua dimensi dengan cara membagi indeks dengan  $N$  untuk mendapatkan baris dan menggunakan operasi *modulo*  $N$  untuk mendapatkan kolom. Hasil konversi ini disimpan dalam `vector` pasangan koordinat yang merepresentasikan posisi *queen* pada papan.

Setelah konfigurasi posisi *queen* terbentuk, fungsi `isValid()` dipanggil untuk memeriksa apakah konfigurasi tersebut memenuhi seluruh aturan, seperti tidak adanya konflik baris, kolom, warna maupun posisi yang tidak diperbolehkan. Jika konfigurasi dinyatakan valid, maka fungsi langsung mengembalikan posisi tersebut sebagai solusi dan proses pencarian dihentikan.

Apabila konfigurasi tidak valid, maka fungsi `nextCombination()` dipanggil untuk menghasilkan kombinasi berikutnya. Proses ini akan terus berulang sampai solusi ditemukan atau seluruh kemungkinan kombinasi telah habis diperiksa. Jika seluruh kombinasi telah diuji dan tidak ada yang memenuhi syarat maka fungsi akan mengembalikan `vector` kosong yang menandakan bahwa tidak terdapat solusi untuk papan tersebut.

8. `printTiles()`

```

PROCEDURE printTiles(isiFile, QueenPositions, N)

    CREATE matrix hasQueen[N][N] initialized to FALSE

    FOR each (r, c) IN QueenPositions DO
        hasQueen[r][c] ← TRUE
    ENDFOR

    FOR i ← 0 TO N - 1 DO
        FOR j ← 0 TO N - 1 DO
            IF hasQueen[i][j] = TRUE THEN
                PRINT "#"
            ELSE
                PRINT isiFile[i][j]
            ENDIF
        ENDFOR
        PRINT newline
    ENDFOR

ENDPROCEDURE

```

Fungsi ini bertugas menampilkan papan ke layar dengan menandai posisi *queen* menggunakan simbol `#`. Implementasinya membuat struktur sementara untuk menandai posisi *queen* agar proses pencetakan lebih mudah. Kemudian dilakukan iterasi terhadap seluruh sel papan dan dicetak sesuai kondisi.



9. saveSolutionToFile()

```
PROCEDURE saveSolutionToFile(isiFile, solusi, N, outputFilename)

    CREATE matrix hasQueen[N][N] initialized to FALSE

    FOR each (r, c) IN solusi DO
        hasQueen[r][c] ← TRUE
    ENDFOR

    OPEN outputFile with outputFilename

    IF outputFile not open THEN
        PRINT error message
        RETURN
    ENDIF

    FOR i ← 0 TO N - 1 DO
        FOR j ← 0 TO N - 1 DO
            IF hasQueen[i][j] = TRUE THEN
                WRITE "#"
            ELSE
                WRITE isiFile[i][j]
            ENDIF
        ENDFOR
        WRITE newline
    ENDFOR

    CLOSE outputFile
    PRINT success message

ENDPROCEDURE
```

Fungsi ini digunakan untuk menyimpan konfigurasi solusi yang telah ditemukan ke dalam sebuah file output. Fungsi ini dipanggil ketika algoritma berhasil menemukan posisi queen yang valid dan pengguna memilih untuk menyimpan hasil tersebut. Tujuan utama dari fungsi ini adalah agar solusi tidak hanya ditampilkan di layar, tetapi juga dapat disimpan secara permanen.

Pada awal fungsi, dibuat sebuah matriks boolean dua dimensi dengan ukuran  $N \times N$  yang digunakan untuk menandai posisi *queen* pada papan. Seluruh elemen matriks ini diinisialisasi dengan nilai *false*. Kemudian, untuk setiap pasangan koordinat (baris, kolom) yang terdapat dalam *vector* solusi, elemen matriks pada posisi tersebut diubah menjadi *true*. Matriks ini berfungsi sebagai penanda agar proses penulisan ke *file* menjadi lebih sederhana dan terstruktur.

Setelah matriks penanda dibuat, fungsi mencoba membuka *file output* menggunakan *ofstream*. Jika *file* tidak dapat dibuka atau dibuat, maka fungsi akan menampilkan pesan kesalahan dan menghentikan proses penyimpanan. Namun jika *file* berhasil dibuka proses penulisan dilakukan baris demi baris sesuai ukuran papan.

Untuk setiap sel pada papan, fungsi akan memeriksa apakah posisi tersebut ditandai sebagai queen dalam matriks *boolean*. Jika ya, maka simbol # akan dituliskan ke dalam file sebagai representasi *queen*. Jika tidak, maka karakter asli dari papan akan dituliskan. Setelah satu baris selesai ditulis, fungsi menambahkan karakter *newline* agar format papan tetap terjaga.

Setelah seluruh isi papan selesai ditulis, *file* kemudian ditutup untuk memastikan seluruh data tersimpan dengan benar. Secara keseluruhan, fungsi ini bekerja dengan membuat ulang papan berdasarkan solusi yang ditemukan, lalu menuliskannya kembali dalam format yang sama seperti tampilan di layar sehingga hasil *file* konsisten dengan output program.

10. main()

```
FUNCTION main() → integer

    WHILE TRUE DO

        CALL refreshScreen

        DECLARE inputFile
        PRINT "Masukkan nama file : "
        INPUT inputFile
```

```

namaFileInput ← "../test/" + inputFile

DECLARE isiFile
DECLARE N

CALL ReadFileToArray(namaFileInput, isiFile, &N)

DECLARE setHuruf as empty set

FOR each line IN isiFile DO
    FOR each c IN line DO
        INSERT c INTO setHuruf
    ENDFOR
ENDFOR

jumlahWarna ← size of setHuruf

iterasiCount ← 0

PRINT "Ukuran papan: ", N, "x", N
PRINT "Jumlah warna: ", jumlahWarna
PRINT "Jumlah kombinasi yang akan dicoba: ",
    Combination(N × N, jumlahWarna)

PRINT "Tekan ENTER untuk mulai..."
WAIT for ENTER

start ← current time

solusi ← bruteForceAlgorithm(
    isiFile,
    N,
    jumlahWarna,
    iterasiCount
)

end ← current time
duration ← end - start (in milliseconds)

PRINT separator line

IF solusi is not empty THEN
    PRINT "Solusi Ditemukan!"
    CALL printTiles(isiFile, solusi, N)
ELSE
    PRINT "Solusi Tidak ada"
ENDIF

PRINT "Waktu pencarian: ", duration
PRINT "Banyak kasus yang ditinjau: ", iterasiCount
PRINT separator line

IF solusi is not empty THEN

    PRINT "Apakah Anda ingin menyimpan solusi? (y/n): "
    INPUT jawab

    IF jawab = 'y' OR jawab = 'Y' THEN

        dotPos ← posisi terakhir karakter '.'
            dalam namaFileInput

        IF dotPos ditemukan THEN
            baseName ← substring namaFileInput
                dari awal sampai sebelum dotPos
        ELSE
            baseName ← namaFileInput
        ENDIF

        outputFilename ← baseName + "_solution.txt"

        CALL saveSolutionToFile(
            isiFile,
            solusi,
            N,
            outputFilename

```

```

    )
ENDIF

ENDIF

PRINT "Apakah Anda ingin mencoba test file yang lain, atau quit?"
PRINT "1. Lanjut"
PRINT "2. Quit"

PRINT "Pilihan : "
INPUT pilihan

IF pilihan = 2 THEN
    PRINT "Program Selesai. Terima Kasih!"
    BREAK
ENDIF

ENDWHILE

RETURN 0

ENDFUNCTION

```

Setelah seluruh fungsi pendukung dijelaskan, fungsi `main()` berperan sebagai pengatur alur program. Fungsi ini menangani interaksi pengguna, pembacaan *file*, pemanggilan algoritma *brute force*, pengukuran waktu eksekusi menggunakan `chrono`, serta opsi penyimpanan solusi.

#### IV. Hasil Pengujian dan Analisis Test Case

##### A. Ringkasan Hasil Test Case

| File    | Ukuran | Total Kombinasi | Iterasi Solusi | Waktu (ms) | % Ruang dicari |
|---------|--------|-----------------|----------------|------------|----------------|
| tc1.txt | 4x4    | 1.820           | 741            | 7          | 40,7           |
| tc2.txt | 5x5    | 53.130          | 8.097          | 16         | 15,2           |
| tc3.txt | 6x6x   | 1.947.792       | 251.215        | 25         | 12,9           |
| tc4.txt | 7x7    | 85.900.584      | 10.075.337     | 213        | 11,7           |
| tc5.txt | 9x9    | ~157 miliar     | ~157 miliar    | 32108967   | ~100%          |

##### B. Test Case 1

```

Masukkan nama file : tc1.txt
Ukuran papan: 4x4
Jumlah warna: 4
Jumlah kombinasi yang akan dicoba: 1820

Tekan ENTER untuk mulai...

Solusi Ditemukan di Iterasi- 741!

=====
Solusi Ditemukan!
A # B B
A A B #
# C D D
C C # D

Waktu pencarian: 7 ms
Banyak kasus yang ditinjau: 741 kasus
=====

Apakah Anda ingin menyimpan solusi? (y/n): █

```

Dengan ukuran papan 4×4 dan 4 daerah warna (A, B, C, D), total kombinasi yang perlu dievaluasi adalah  $C(16,4) = 1.820$ . Solusi ditemukan pada iterasi ke-741, yang berarti program harus menelusuri sekitar 40,7% dari keseluruhan ruang pencarian sebelum menemukan konfigurasi yang valid. Waktu eksekusi sebesar 7 ms tergolong sangat cepat, menandakan bahwa algoritma brute force masih sangat efisien untuk ukuran papan sekecil ini.

### C. Test Case 2

```
Masukkan nama file : tc2.txt
Ukuran papan: 5x5
Jumlah warna: 5
Jumlah kombinasi yang akan dicoba: 53130

Tekan ENTER untuk mulai...

Solusi Ditemukan di Iterasi- 8097!

=====
Solusi Ditemukan!
# A A B B
A A # B B
D C C B #
D # C E E
D D D # E

Waktu pencarian: 16 ms
Banyak kasus yang ditinjau: 8097 kasus
=====

Apakah Anda ingin menyimpan solusi? (y/n): █
```

Peningkatan ukuran papan menjadi 5×5 mengakibatkan lonjakan signifikan pada total kombinasi, yaitu  $C(25,5) = 53.130$ , atau sekitar 29 kali lebih banyak dibandingkan test case sebelumnya. Meskipun demikian, solusi berhasil ditemukan pada iterasi ke-8.097, yang hanya mencakup 15,2% dari total ruang pencarian. Waktu eksekusi 16 ms masih sangat baik dan menunjukkan bahwa program mampu menangani papan berukuran sedang dengan performa yang memadai.

### D. Test Case 3

```
Masukkan nama file : tc3.txt
Ukuran papan: 6x6
Jumlah warna: 6
Jumlah kombinasi yang akan dicoba: 1947792

Tekan ENTER untuk mulai...

Solusi Ditemukan di Iterasi- 251215!

=====
Solusi Ditemukan!
# A A B B B
A A A # B B
C # C D D D
C C C D # D
E E # F F F
E E E F F #

Waktu pencarian: 25 ms
Banyak kasus yang ditinjau: 251215 kasus
=====

Apakah Anda ingin menyimpan solusi? (y/n): █
```

Pada papan 6×6 dengan 6 region warna, total kombinasi mencapai  $C(36,6) = 1.947.792$ , sebuah peningkatan yang sangat drastis dibandingkan ukuran sebelumnya. Solusi ditemukan pada iterasi ke-251.215, setara dengan 12,9% dari total ruang pencarian.

### E. Test Case 4

```
Solusi Ditemukan di Iterasi- 157030560983!

=====
Solusi Ditemukan!
A A A B B C C # D
A B B B # C E C D
A B B B D C # C D
A # A B D C C C D
B B B B D # D D D
F G G # D D H D D
# G I G D D H D D
F G # G D D H D D
F G G D D H H #

Waktu pencarian: 3210967 ms
Banyak kasus yang ditinjau: 157030560983 kasus
=====

Apakah Anda ingin menyimpan solusi? (y/n): █
```

Papan  $7 \times 7$  dengan 7 warna menghadirkan kompleksitas yang jauh lebih tinggi dengan  $C(49,7) = 85.900.584$  kombinasi. Solusi ditemukan pada iterasi ke-10.075.337, mencakup sekitar 11,7% dari total ruang pencarian. Waktu eksekusi sebesar 213 ms sudah mulai terasa lebih lama, namun masih tergolong dapat diterima untuk ukuran input ini. Hasil ini memperlihatkan tren yang konsisten, yaitu seiring bertambahnya ukuran papan, jumlah kombinasi tumbuh secara eksponensial, namun persentase ruang pencarian yang perlu ditelusuri cenderung menurun.

#### F. Test Case 5

```
Masukkan nama file : tc5.txt
Ukuran papan: 7x7
Jumlah warna: 7
Jumlah kombinasi yang akan dicoba: 85900584

Tekan ENTER untuk mulai...

Solusi Ditemukan di Iterasi- 10075337!

=====
Solusi Ditemukan!
# A A B B B
A A A B # B
C # C D B B
C C C # D E E
C C C F F E #
G G # F F E E
G G G F # E E

Waktu pencarian: 213 ms
Banyak kasus yang ditinjau: 10075337 kasus

Apakah Anda ingin menyimpan solusi? (y/n):
```

Test case terakhir menggunakan konfigurasi papan  $9 \times 9$  dari soal asli tugas, yang memiliki total kombinasi sebesar  $C(81,9) \approx 157$  miliar. Ini merupakan test case terberat yang diuji, dan hasilnya menunjukkan keterbatasan fundamental dari pendekatan brute force murni: waktu eksekusi mencapai 3.210.967 ms atau sekitar 53 menit. Solusi baru ditemukan setelah menelusuri hampir seluruh ruang pencarian (157.030.560.983 iterasi). Hasil ini secara empiris membuktikan bahwa kompleksitas waktu algoritma brute force bersifat kombinatorial, sehingga tidak *scalable* untuk input besar.

#### V. Kesimpulan

Berdasarkan perancangan dan implementasi yang telah dilakukan, algoritma *brute force* pada tugas ini berhasil menyelesaikan permasalahan *Queens* sesuai dengan seluruh *constraint* yang diberikan, yaitu tidak adanya konflik pada baris, kolom, warna, serta aturan larangan bersebelahan. Seluruh test case yang memiliki solusi dapat ditemukan dengan konfigurasi yang valid sehingga dapat disimpulkan bahwa mekanisme pembangkitan kombinasi dan proses validasi telah berjalan dengan benar.

Dari hasil pengujian terlihat bahwa kompleksitas algoritma bersifat kombinatorial, yaitu bergantung pada nilai  $C(N^2, K)$ . Untuk ukuran papan kecil hingga menengah, waktu eksekusi masih relatif cepat dan dapat diterima. Namun, ketika ukuran papan semakin besar, jumlah kombinasi meningkat secara sangat signifikan sehingga waktu eksekusi bertambah drastis. Pada papan berukuran besar, hampir seluruh ruang pencarian harus diperiksa sebelum solusi ditemukan yang menunjukkan keterbatasan pendekatan brute force murni.

Meskipun demikian, pengerjaan tugas ini memberikan pembelajaran yang sangat baik dalam memahami bagaimana ruang solusi tumbuh secara kombinatorial dan bagaimana performa algoritma dipengaruhi oleh kompleksitas waktu. Melalui implementasi brute force secara langsung, saya dapat melihat secara konkret hubungan antara teori kompleksitas dan hasil eksekusi di lapangan. Secara keseluruhan, metode brute force efektif untuk memastikan kebenaran solusi, namun dari sisi efisiensi jelas bukan pendekatan yang paling optimal untuk permasalahan skala besar.

#### VI. Pranala Repository

Link Github : [https://github.com/fahdmz/Tucil1\\_13524078](https://github.com/fahdmz/Tucil1_13524078)

VII. Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Fahd Muhammad Zahid

VIII. Tabel Ceklis

| No | Point                                                                              | Ya | Tidak |
|----|------------------------------------------------------------------------------------|----|-------|
| 1  | Program berhasil di kompilasi tanpa kesalahan                                      | ✓  |       |
| 2  | Program berhasil di jalankan                                                       | ✓  |       |
| 3  | Solusi yang diberikan program benar dan mematuhi aturan permainan                  | ✓  |       |
| 4  | Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt | ✓  |       |
| 5  | Program memiliki Graphical User Interface (GUI)                                    |    | ✓     |
| 6  | Program dapat menyimpan solusi dalam bentuk file gambar                            |    | ✓     |

